TypeScript is a popular choice for programmers accustomed to other languages with static typing, such as C# and Java.

TypeScript's type system offers many of the same benefits, such as better code completion, earlier detection of errors, and clearer communication between parts of your program. While TypeScript provides many familiar features for these developers, it's worth stepping back to see how JavaScript (and therefore TypeScript) differ from traditional OOP languages. Understanding these differences will help you write better JavaScript code, and avoid common pitfalls that programmers who go straight from C#/Java to TypeScript may fall into.

Co-learning JavaScript

If you're familiar with JavaScript already but are primarily a Java or C# programmer, this introductory page can help explain some of the common misconceptions and pitfalls you might be susceptible to. Some of the ways that TypeScript models types are quite different from Java or C#, and it's important to keep these in mind when learning TypeScript.

If you're a Java or C# programmer that is new to JavaScript in general, we recommend learning a little bit of JavaScript *without* types first to understand JavaScript's runtime behaviors. Because TypeScript doesn't change how your code *runs*, you'll still have to learn how JavaScript works in order to write code that actually does something!

It's important to remember that TypeScript uses the same *runtime* as JavaScript, so any resources about how to accomplish specific runtime behavior (converting a string to a number, displaying an alert, writing a file to disk, etc.) will always apply equally well to TypeScript programs. Don't limit yourself to TypeScript-specific resources!

Rethinking the Class

C# and Java are what we might call *mandatory OOP* languages. In these languages, the *class* is the basic unit of code organization, and also the basic container of all data *and* behavior at runtime. Forcing all functionality and data to be held in classes can be a good domain model for some problems, but not every domain *needs* to be represented this way.

Free Functions and Data

In JavaScript, functions can live anywhere, and data can be passed around freely without being inside a pre-defined class or struct. This flexibility is extremely powerful. "Free" functions (those not associated with a class) working over data without an implied OOP hierarchy tend to be the preferred model for writing programs in JavaScript.

Static Classes

Additionally, certain constructs from C# and Java such as singletons and static classes are unnecessary in TypeScript.

OOP in TypeScript

That said, you can still use classes if you like! Some problems are well-suited to being solved by a traditional OOP hierarchy, and TypeScript's support for JavaScript classes will

make these models even more powerful. TypeScript supports many common patterns such as implementing interfaces, inheritance, and static methods.

We'll cover classes later in this guide.

Rethinking Types

TypeScript's understanding of a *type* is actually quite different from C# or Java's. Let's explore some differences.

Nominal Reified Type Systems

In C# or Java, any given value or object has one exact type - either null, a primitive, or a known class type. We can call methods like value.GetType() or value.getClass() to query the exact type at runtime. The definition of this type will reside in a class somewhere with some name, and we can't use two classes with similar shapes in lieu of each other unless there's an explicit inheritance relationship or commonly-implemented interface.

These aspects describe a *reified, nominal* type system. The types we wrote in the code are present at runtime, and the types are related via their declarations, not their structures.

Types as Sets

In C# or Java, it's meaningful to think of a one-to-one correspondence between runtime types and their compile-time declarations.

In TypeScript, it's better to think of a type as a *set of values* that share something in common. Because types are just sets, a particular value can belong to *many* sets at the same time.

Once you start thinking of types as sets, certain operations become very natural. For example, in C#, it's awkward to pass around a value that is *either* a string or int, because there isn't a single type that represents this sort of value.

In TypeScript, this becomes very natural once you realize that every type is just a set. How do you describe a value that either belongs in the string set or the number set? It simply belongs to the *union* of those sets: string | number.

TypeScript provides a number of mechanisms to work with types in a set-theoretic way, and you'll find them more intuitive if you think of types as sets.

Erased Structural Types

In TypeScript, objects are *not* of a single exact type. For example, if we construct an object that satisfies an interface, we can use that object where that interface is expected even though there was no declarative relationship between the two.

```
interface Pointlike {
  x: number;
  y: number;
}
```

```
interface Named {
  name: string;
}

function logPoint(point: Pointlike) {
  console.log("x = " + point.x + ", y = " + point.y);
}

function logName(x: Named) {
  console.log("Hello, " + x.name);
}

const obj = {
  x: 0,
  y: 0,
  name: "Origin",
};

logPoint(obj);
logName(obj);
```

Try

TypeScript's type system is *structural*, not nominal: We can use obj as a Pointlike because it has x and y properties that are both numbers. The relationships between types are determined by the properties they contain, not whether they were declared with some particular relationship.

TypeScript's type system is also *not reified*: There's nothing at runtime that will tell us that obj is Pointlike. In fact, the Pointlike type is not present *in any form* at runtime.

Going back to the idea of *types as sets*, we can think of obj as being a member of both the Pointlike set of values and the Named set of values.

Consequences of Structural Typing

OOP programmers are often surprised by two particular aspects of structural typing.

Empty Types

The first is that the *empty type* seems to defy expectation:

```
class Empty {}


function fn(arg: Empty) {
  // do something?
}


// No error, but this isn't an 'Empty' ?
fn({ k: 10 });
```

Try

TypeScript determines if the call to fn here is valid by seeing if the provided argument is a valid Empty. It does so by examining the *structure* of { k: 10 } and class Empty { }. We can see that { k: 10 } has *all* of the properties that Empty does, because Empty has no properties. Therefore, this is a valid call!

This may seem surprising, but it's ultimately a very similar relationship to one enforced in nominal OOP languages. A subclass cannot *remove* a property of its base class, because doing so would destroy the natural subtype relationship between the derived class and its base. Structural type systems simply identify this relationship implicitly by describing subtypes in terms of having properties of compatible types.

Identical Types

Another frequent source of surprise comes with identical types:

```
class Car {
  drive() {
    // hit the gas
  }
}
class Golfer {
  drive() {
    // hit the ball far
  }
}
// No error?
```

```
let w: Car = new Golfer();
```

Again, this isn't an error because the *structures* of these classes are the same. While this may seem like a potential source of confusion, in practice, identical classes that shouldn't be related are not common.

We'll learn more about how classes relate to each other in the Classes chapter.

Reflection

OOP programmers are accustomed to being able to query the type of any value, even a generic one:

```
// C#
static void LogType<T>() {
  Console.WriteLine(typeof(T).Name);
}
```

Because TypeScript's type system is fully erased, information about e.g. the instantiation of a generic type parameter is not available at runtime.

JavaScript does have some limited primitives like typeof and instanceof, but remember that these operators are still working on the values as they exist in the type-erased output code. For example, typeof (new Car()) will be "object", not Car or "Car".

Next Steps

This was a brief overview of the syntax and tools used in everyday TypeScript. From here, you can:

- Read the full Handbook from start to finish
- Explore the Playground examples